

Развёртывание системы BHS.CRG

Система исполнительной документации BHS.CRG

Версия документа: 2026-08-21

Инструкция по развёртыванию BHS.CRG

Система генерации исполнительной документации для электромонтажных проектов. Развёртывание — через **Docker Compose** на сервере **Linux**: весь стек (веб-интерфейс, сервер приложений, база данных, объектное хранилище, модель распознавания) поднимается несколькими командами.

Этот документ — для администратора/инженера, разворачивающего систему. Для работы с системой см. «Инструкцию пользователя» и «Инструкцию администратора».

1. Архитектура развёртывания

Компонент	Образ / технология	Назначение
web	nginx + собранный React-SPA	Веб-интерфейс; проксирует /api на сервер
api	ASP.NET Core 10 + Typst	Сервер приложений, REST API, генерация PDF
postgres	PostgreSQL 16	Основная база данных
minio	MinIO	Объектное хранилище (сканы, наборы данных, готовые файлы)
ollama	Ollama	Локальная модель распознавания сканов (опционально)

Вспомогательные одноразовые сервисы: `createbuckets` (создаёт bucket в MinIO), `ollama-init` (загружает модель распознавания).

Схематично:



Генерация: PDF собирается движком **Typst** (входит в образ `api`). Формат **DOCX в текущей версии не поддерживается** — только PDF.

2. Требования

- **HTTPS обязателен.** Система работает с паролями и токенами доступа; по открытому HTTP они идут по сети в читаемом виде, и перехват одного запроса даёт полный доступ к системе от имени пользователя. Стек TLS не терминирует сам — перед сервисом `web` ставится обратный прокси с сертификатом (nginx, Caddy, Traefik) либо терминатор периметра. Без него допустима только установка, целиком закрытая внутри доверенной сети. См. §11.
- **ОС:** Linux (Ubuntu 22.04+/Debian 12+ и совместимые).
- **Docker Engine 24+** и **Docker Compose v2** (`docker compose`, не `docker-compose`).
- **Ресурсы:**
 - Базовый стек (без Ollama): от **2 CPU / 4 ГБ RAM / 10 ГБ диска**.

- С Ollama и моделью `qwen2.5vl:7b` : дополнительно **~8 ГБ RAM** и **~6 ГБ диска** под модель; желательна видеокарта, но работает и на CPU (медленнее).
- Доступ в интернет на этапе сборки (загрузка образов, Typst, npm/NuGet-пакетов) и, при использовании веб-поиска документов качества, в рантайме.

Проверка окружения:

```
docker --version
docker compose version
```

3. Быстрый старт

Исходный код для установки **не нужен**: `api` и `web` поставляются готовыми образами из GHCR. Достаточно двух файлов со страницы выпуска.

```
# 1. Взять со страницы выпуска (https://github.com/pavru/bhs.crg/releases/latest)
#   docker-compose.yml и .env.example – они приложены к релизу.
mkdir bhs-crg && cd bhs-crg
curl -LO https://github.com/pavru/bhs.crg/releases/latest/download/docker-compose.yml
curl -Lo .env.example https://github.com/pavru/bhs.crg/releases/latest/download/.env.example

# 2. Подготовить конфигурацию
cp .env.example .env
nano .env                # задать пароли, JWT-секрет и APP_VERSION (см. раздел 4)

# 3. Запустить весь стек (образы скачаются из GHCR)
docker compose up -d

# 4. Проверить статус
docker compose ps
docker compose logs -f api
```

Дальше в инструкции команды пишутся в виде `docker compose -f deploy/docker-compose.yml ...` — это форма для запуска из клона репозитория. Если вы поставили систему из релиза, файл лежит рядом, и `-f deploy/docker-compose.yml` можно опускать.

Сборка из исходников (разработка, проверка правки до релиза, установка без доступа к ghcr.io):

```
git clone https://github.com/pavru/bhs.crg.git && cd bhs.crg
cp deploy/.env.example deploy/.env && nano deploy/.env
docker compose -f deploy/docker-compose.yml -f deploy/docker-compose.build.yml up -d --build
```

Образы публичные — `docker login ghcr.io` для установки не нужен. Если `pull` требует авторизации, значит пакет в GitHub остался приватным (так создаётся первый): владельцу репозитория надо один раз открыть его в *Packages* → *Package settings* → *Change visibility*.

После запуска веб-интерфейс доступен по адресу **`http://СЕРВЕР:8080`** (порт задаётся `WEB_PORT` в `.env`).

При первом старте сервер `api` автоматически применяет миграции базы данных и создаёт роли `Admin / User`. Отдельных команд миграции выполнять не нужно.

4. Конфигурация (`deploy/.env`)

Скопируйте `deploy/.env.example` → `deploy/.env` и заполните. Файл `.env` содержит секреты и **не** хранится в репозитории.

Переменная	Назначение	Обязательно
POSTGRES_DB / POSTGRES_USER / POSTGRES_PASSWORD	Параметры БД. Пароль из примера останавливает запуск	да (сменить пароль)
MINIO_ROOT_USER / MINIO_ROOT_PASSWORD	Учётные данные MinIO — это администратор хранилища . Незаданные или взятые из примера останавливают запуск	да (сменить оба)
MINIO_BUCKET	Имя bucket для файлов. Пустое значение останавливает запуск	да (по умолч. <code>bhs-crg</code>)
MINIO_CONSOLE_PORT / MINIO_CONSOLE_BIND	Порт и адрес публикации веб-консоли MinIO. По умолчанию <code>127.0.0.1</code> — только с самого сервера, см. §6	нет
JWT_KEY	Секрет подписи токенов, ≥ 32 символов	да
WEB_PORT	Порт веб-интерфейса на хосте	нет (по умолч. <code>8080</code>)
FORWARDED_TRUSTED_NETWORKS	Сети, чьему заголовку <code>X-Forwarded-For</code> сервер доверяет (CIDR через запятую). Пусто — петля и частные диапазоны. Негодная запись останавливает запуск	нет
APP_PUBLIC_URL	Публичный адрес системы (напр. <code>https://docs.example.ru</code>) — подставляется ссылкой в письма	да, если нужна почта
BACKUP_MAX_ARCHIVE_MB	Предел размера резервной копии, МБ (16...10240). Ей же задаётся <code>client_max_body_size</code> веб-сервера — второе значение править не нужно. Негодное останавливает запуск	нет (по умолч. <code>500</code>)
OLLAMA_MODEL	Модель распознавания	нет (по умолч. <code>qwen2.5vl:7b</code>)
ANTHROPIC_API_KEY	Распознавание реквизитов (Claude)	опц.
GEMINI_API_KEY	Распознавание реквизитов (Gemini)	опц.
SERPER_API_KEY	Веб-поиск документов качества	опц.

Сгенерировать надёжный `JWT_KEY` :

```
openssl rand -base64 48
```

Важно: обязательно смените пароли БД/MinIO и `JWT_KEY` перед вводом в эксплуатацию. Значения из примера — только для ознакомления.

`JWT_KEY` проверяется при запуске: **приложение не поднимется**, если значение не задано, короче 32 байт или оставлено из файла-примера. Это сделано намеренно — иначе установку легко ввести в эксплуатацию, не заметив, что ключ подписи не менялся.

Так же проверяются настройки хранилища и параметры БД. Раньше при незадаанных значениях система поднималась молча, а отказ приходил позже и чужими словами: ошибка клиента MinIO при первой загрузке файла, `Host can't be null` от драйвера БД. Теперь запуск останавливается с указанием, что именно задать, если не заполнено любое из:

- `MINIO_ROOT_USER` , `MINIO_ROOT_PASSWORD` , `MINIO_BUCKET` ;
- **любая часть** строки подключения — `POSTGRES_DB` , `POSTGRES_USER` , `POSTGRES_PASSWORD` или адрес сервера. Строка разбирается, а не проверяется на пустоту целиком: незаданная переменная даёт не пустую строку, а строку с пустым полем (`...;Password=`), и проверка «не пусто» такое пропустила бы — то есть ровно в том случае, с которым и сталкиваются, её бы не было.

Сюда же — `BACKUP_MAX_ARCHIVE_MB` : не число или значение вне 16...10240 останавливает запуск. Понять «500 МБ» как 500 и промолчать было бы хуже, чем отказать: описка в `.env` тихо вернула бы умолчание на установке, где предел специально поднимали. Контейнер `web` на негодном значении тоже не поднимается.

Значение, оставленное из файла-примера, тоже останавливает запуск. Смотреть `docker compose logs api` .

Пароль БД обязателен: вход без пароля (`trust` , `.pgpass`) наша поставка не разворачивает, а СУБД, доступная по сети без пароля, — сама по себе дефект.

Если система уже работала на значении из примера, после смены ключа **отзовите refresh-токены**: выданные ранее перестанут быть доверенными, пользователи войдут заново. Это ожидаемое поведение.

Почтовые сценарии (сброс пароля, подтверждение email)

Письма со **ссылками** — самостоятельный сброс пароля, подтверждение адреса, смена email, отправка крупного комплекта — требуют **двух** настроек:

1. **APP_PUBLIC_URL** в `deploy/.env` — внешний адрес, по которому пользователи открывают систему (именно он подставляется в ссылку письма). Без него письма со ссылками **не отправляются**, а действие возвращает понятную ошибку (пользователь бесполезного письма не получит).
2. **SMTP** — параметры почтового сервера задаёт администратор в UI: **Настройка системы** → **Настройки** → **Почта** (host/port/логин/шифрование). Без SMTP письма не уходят.

Ключи шифрования одноразовых токенов (сброс/подтверждение) хранятся на томе `dp_keys` (в `compose` уже настроен) — поэтому ссылки переживают перезапуск контейнера `api` . Срок жизни access-токена — 60 мин (обновляется автоматически); настраивается `Jwt__AccessTokenMinutes` при необходимости.

5. Первый вход (создание администратора)

1. Откройте **`http://SERVER:8080`**.
2. Пока в системе нет ни одного пользователя, доступна **регистрация первого администратора**. Зарегистрируйте учётную запись — она автоматически получит роль **Администратор**.

3. После этого **публичная регистрация закрывается**. Все остальные пользователи создаются администратором в разделе **Настройка системы** → **Пользователи** (см. «Инструкцию администратора»).

6. Порты

Сервис	Внутренний порт	Порт на хосте	Примечание
web (nginx)	8080	<code>WEB_PORT</code> (8080)	Основной вход в систему
api	8080	—	Доступен только внутри сети Compose
postgres	5432	—	Наружу не публикуется
minio (API)	9000	—	Наружу не публикуется
minio (консоль)	9001	<code>127.0.0.1:MINIO_CONSOLE_PORT</code>	Админ-консоль MinIO, только с самого сервера
ollama	11434	—	Наружу не публикуется

Наружу выведен только веб-интерфейс. Базу, хранилище и Ollama публиковать не следует.

Консоль MinIO публикуется на петлю (`MINIO_CONSOLE_BIND` , по умолчанию `127.0.0.1`): она открывает управление хранилищем под учётной записью `MINIO_ROOT_USER` , то есть второй вход ко всем файлам системы в обход приложения и его ролей. С рабочей машины на неё ходят через SSH-туннель:

```
ssh -L 9001:127.0.0.1:9001 пользователь@сервер # затем http://localhost:9001
```

Внутренний порт `web` — 8080, а не 80: nginx в образе работает от непривилегированного пользователя. Порт на хосте (`WEB_PORT`) прежний, для обратного прокси ничего не меняется.

Заголовки безопасности

Сервис `web` отдаёт `Content-Security-Policy` , `X-Content-Type-Options` , `X-Frame-Options` , `Referrer-Policy` , `Permissions-Policy` и `Strict-Transport-Security` (см. `deploy/nginx.conf`). Политика содержимого разрешает скрипты **только с адреса самого приложения** — сторонние CDN и внедрённые в данные инлайновые скрипты не выполняются.

Если вы правите фронтенд под себя и подключаете стороннюю библиотеку по ссылке, она будет заблокирована — это ожидаемо. Правильный путь: положить библиотеку в сборку, а не ослаблять политику. HSTS браузер применяет только поверх HTTPS, поэтому заголовок безвреден до появления терминатора TLS и включается сам, когда тот появится.

7. Опциональные компоненты

Распознавание сканов (документы качества)

- **Ollama (локально):** модель загружается сервисом `ollama-init` при первом `up`. Загрузить/обновить вручную:

```
docker compose -f deploy/docker-compose.yml exec ollama ollama pull qwen2.5vl:7b
```

- **Облачные модели:** задайте `ANTHROPIC_API_KEY` и/или `GEMINI_API_KEY` в `.env`. Порядок перебора движков настраивается в коде (`Recognition:Order`) и в разделе настроек интеграций.

Веб-поиск документов качества

- Задайте `SERPER_API_KEY` (`serper.dev`) в `.env`. Без ключа поиск в интернете недоступен, остальная работа с документами качества — да.

Ключи интеграций можно также задавать/менять в самом приложении: **Настройка системы** → **Настройки** → **Поиск и распознавание**. В базе они хранятся **зашифрованными** (ключами Data Protection с тома `dp_keys`) и маскируются при чтении через API. Ключи, заданные переменными окружения, шифровать нечем и не нужно — они и так вне базы.

Следствие: при потере тома `dp_keys` расшифровать сохранённые ключи и пароль SMTP не получится — их надо будет ввести заново. Пока том просто недоступен (не примонтирован, переехал путь), сохранённые значения не портятся: приложение покажет их как незаданные, но перезаписывать не станет — вернёте том, и они снова читаются.

Пароль SMTP привязан к серверу. Смена хоста, порта, пользователя или снятие шифрования обнуляют сохранённый пароль: его надо ввести заново. Это не придирка — иначе сохранение чужого адреса с пустым полем пароля отправило бы туда сохранённый пароль с первым же письмом. По той же причине проверка связи с другим сервером требует ввести пароль явно.

8. Обновление системы

Обновление — это смена номера версии в `.env` и подтягивание образов.


```
# 1. РЕЗЕРВНАЯ КОПИЯ – до всего остального (раздел 9). Схема базы мигрирует при старте новой
# версии, и отката миграций нет: вернуть прошлый образ можно, прошлую схему – только из копии.

# 2. Посмотреть, что вышло: https://github.com/pavru/bhs.crg/releases

# 3. Обновить compose-файл и сверить шаблон окружения – они приложены к каждому выпуску.
# Новые версии добавляют в них переменные, тома и лимиты; обновив только образы, вы получите
# новую настройку в её умолчании и не узнаете об этом.
curl -LO https://github.com/pavru/bhs.crg/releases/latest/download/docker-compose.yml
curl -Lo .env.example.new https://github.com/pavru/bhs.crg/releases/latest/download/.env.example
diff .env.example.new .env.example      # что добавилось – перенести в свой .env

# 4. Указать версию и обновиться
nano .env                                # APP_VERSION=X.Y.Z
docker compose pull
docker compose up -d

# 5. Проверить
docker compose ps
curl -s http://localhost:8080/api/version
```

Анонимный ответ `/api/version` показывает только номер версии: хеш коммита и дату сборки система отдаёт вошедшему пользователю — они видны внизу боковой панели интерфейса (`v0.136.0 · a1b2c3d`, дата сборки в подсказке). Пустое поле `commit` в ответе `curl` — не признак неправильно собранного образа.

Миграции БД применяются автоматически при старте `api`. Данные сохраняются в томах (`postgres_data`, `minio_data`, `ollama_data`).

Откат. Вернуть прошлый `APP_VERSION` и повторить `pull + up -d` — но только если новая версия не успела применить миграции: старый образ с новой схемой работать не обязан. Если миграции применились, откат делается восстановлением резервной копии (раздел 9).

Разово при переходе на поставку образами (с версии ниже 0.137.0). Прежде `api` и `web` собирались на хосте из исходников (`up -d --build`), теперь тянутся из GHCR. Тома, имена сервисов и данные те же; достаточно задать `APP_VERSION` в `.env` и выполнить `docker compose pull && docker compose up -d`. Локально собранные образы после этого можно удалить (`docker image prune`). Обратный путь — сборка из исходников — остаётся: `-f deploy/docker-compose.yml -f deploy/docker-compose.build.yml up -d --build`.

Разово при обновлении с версии ниже 0.91.0. Контейнеры перестали работать от `root`. Том `dp_keys` (ключи шифрования одноразовых ссылок) создавался от `root`, и владелец у существующего тома сам не поменяется — сервер сможет читать ключи, но не сможет выписать очередной, и через несколько недель ссылки сброса пароля начнут отказывать. Выполните один раз:

```
docker compose -f deploy/docker-compose.yml run --rm --user root api chown -R app:app /app/dp
```

На новой установке делать ничего не нужно: пустой том наследует владельца из образа.

9. Резервное копирование и восстановление

База данных:

```
# Бэкап
docker compose -f deploy/docker-compose.yml exec -T postgres \
  pg_dump -U postgres bhs_crg > backup_$(date +%F).sql

# Восстановление
cat backup_YYYY-MM-DD.sql | docker compose -f deploy/docker-compose.yml exec -T postgres \
  psql -U postgres -d bhs_crg
```

Файлы (MinIO): резервируйте том `minio_data` (или зеркалируйте bucket через `mc mirror`).

В приложении также есть встроенное **резервное копирование** для администратора (**Настройка системы** → **Настройки** → **Резервное копирование**).

10. Диагностика

Симптом	Причина / решение
<code>api</code> падает при старте	Обязательное значение не задано или оставлено из примера — сообщение в логе называет, какое именно и какой переменной задаётся: <code>JWT_KEY</code> , <code>MINIO_ROOT_USER</code> , <code>MINIO_ROOT_PASSWORD</code> , <code>MINIO_BUCKET</code> , параметры БД. Проверьте также доступность <code>postgres</code> (healthcheck). <code>docker compose logs api</code>
Генерация PDF с ошибкой шрифтов	В образ включены DejaVu/Liberation/Noto. Если шаблон требует особый шрифт — добавьте его в <code>Dockerfile.api</code>
Распознавание не работает	Модель Ollama не загружена (<code>ollama pull</code>) или не задан API-ключ. Это опциональная функция
Веб-интерфейс открывается, но «не видит» сервер	Проверьте, что сервис <code>web</code> проксирует на <code>api:8080</code> (см. <code>deploy/nginx.conf</code>)
Вход отвечает «слишком много попыток входа»	Сработал предел частоты по адресу клиента. Если за одним внешним адресом работает целый офис — так и задумано. Если предел ловят единичные пользователи, значит адрес клиента до сервера не доходит: проверьте, что каждый прокси в цепочке дописывает <code>X-Forwarded-For</code> (в поставляемом <code>nginx.conf</code> это <code>\$proxy_add_x_forwarded_for</code>), а сети всех прокси попадают в <code>FORWARDED_TRUSTED_NETWORKS</code> — иначе адресом клиента станет прокси, один на всех
Ссылки сброса пароля перестали работать после обновления	Права на том <code>dp_keys</code> — см. разовую команду в §8
<code>api</code> или <code>ollama</code> перезапускаются без внятной причины, в логах обрыв на тяжёлой операции	Упёрлись в потолок памяти контейнера. Поднимите <code>API_MEMORY_LIMIT</code> / <code>OLLAMA_MEMORY_LIMIT</code> в <code>.env</code> . Проверить: <code>docker stats</code> и <code>docker inspect --format '{{.State.OOMKilled}}'</code> <контейнер>
Вёрстка PDF отвечает «не завершилась за 60 с»	Шаблон зациклился либо рекурсия без выхода. Процесс останавливается намеренно: без срока он занимал бы ядро до перезапуска сервера
Не загружаются крупные файлы	Наибольший предел — архив резервной копии, и задаётся он одной переменной <code>BACKUP_MAX_ARCHIVE_MB</code> : из неё же считается <code>client_max_body_size</code> при старте <code>web</code> (см. <code>deploy/nginx-body-size.sh</code>). Два числа в разных файлах здесь держать нельзя — порог <code>nginx</code> ниже нашего делает предел приложения недостижимым, а отказ приходит HTML-страницей <code>nginx</code> без поля <code>error</code> , и интерфейс показывает голое сообщение <code>axios</code> . Проверить действующее значение: <code>docker compose logs web grep client_max_body_size</code>
Восстановление отвечает «Файл превышает N МБ»	Копия переросла предел. Текущий вес против предела виден в интерфейсе: Настройки → Резервное копирование . Поднимите <code>BACKUP_MAX_ARCHIVE_MB</code> и пересоздайте <code>api</code> и <code>web</code>

Полезные команды:

```
docker compose -f deploy/docker-compose.yml ps          # статус
docker compose -f deploy/docker-compose.yml logs -f api  # логи сервера
docker compose -f deploy/docker-compose.yml down         # остановить (тома сохраняются)
docker compose -f deploy/docker-compose.yml down -v      # остановить и УДАЛИТЬ данные
```

11. Безопасность (чек-лист перед эксплуатацией)

- ☐ Сменены пароли `POSTGRES_PASSWORD`, `MINIO_ROOT_PASSWORD`, а также `MINIO_ROOT_USER` (значение из примера — `minioadmin`).
- ☐ Задан свой `JWT_KEY` (≥ 32 символов, случайный). Без этого приложение не запустится.
- ☐ Заданы свои `MINIO_ROOT_USER` / `MINIO_ROOT_PASSWORD`, `MINIO_BUCKET` и все параметры БД (`POSTGRES_DB` / `POSTGRES_USER` / `POSTGRES_PASSWORD`). Значения из файла-примера приложение тоже не примет — запуск останавливается с указанием переменной.
- ☐ Порты БД/MinIO/Ollama не опубликованы в интернет. Консоль MinIO поставляемый `docker-compose.yml` публикует только на петлю — убедитесь, что `MINIO_CONSOLE_BIND` не переопределён на `0.0.0.0` (см. §6).
- ☐ Том `dp_keys` защищён как секрет: доступ к нему (или к его копии) позволяет выписать действительную ссылку сброса пароля на любую учётную запись, минуя пароль и блокировку. В общие резервные копии его не кладут; если кладут — хранят вместе с прочими секретами. **Он же расшифровывает ключи интеграций** (см. §7): при потере тома ключи распознавания, веб-поиска и пароль SMTP придётся ввести заново — данные и документы при этом не страдают.
- ☐ **HTTPS обязателен**, если система доступна не только с локальной машины: перед `web` стоит обратный прокси с терминацией TLS, порт `80` наружу закрыт. Приложение TLS не терминирует и само на HTTPS не перенаправляет — без внешнего контура пароли и токены идут открытым текстом.
- ☐ Создан администратор; лишние тестовые учётные записи удалены.
- ☐ Настроено регулярное резервное копирование БД и `minio_data`.
- ☐ Известно, где смотреть вес встроенной резервной копии против предела восстановления (**Настройки** → **Резервное копирование**). Вес задаётся библиотекой документов качества и растёт вместе с ней; предел поднимается переменной `BACKUP_MAX_ARCHIVE_MB`.